Federated Learning over Networks: Network Optimization for Decentralized Model Training

Nazlı Ceyda Ünsoy

Abstract

Traditional centralized machine learning requires aggregating data from distributed sources into a central server, creating privacy risks, bandwidth bottlenecks, and latency issues. Federated Learning (FL) addresses these challenges by enabling collaborative model training across multiple edge devices without centralizing raw data. Instead of transmitting datasets, only model parameters are communicated over the network, significantly reducing bandwidth consumption while preserving privacy.

This research investigates the internal mechanisms of Federated Learning, focusing on the Federated Averaging (FedAvg) algorithm and its underlying network communication protocols. The study examines how FL distributes global model parameters to clients, performs local training on decentralized datasets, and aggregates model updates through weighted averaging.

The report will demonstrate why FL is designed as a distributed client-server architecture instead of traditional centralized training by analyzing network overhead, communication rounds, and addressing challenges such as network heterogeneity (varying client bandwidth and latency), non-IID data distribution, and stragglers. A Python-based implementation simulates federated training over raw TCP sockets, examining communication efficiency, convergence speed, and scalability across varying client configurations.

Github Link: https://github.com/ncunsoy/Federated_Learning

Keywords: Federated Learning, Network Protocols, Distributed Systems, Communication Efficiency, Privacy-Preserving Networks

1 Introduction

The rapid increase in smartphones, IoT devices, and edge computing has introduced significant challenges for machine learning systems. Traditional centralized approaches assume that training data can be collected and aggregated on a central server. However, this model faces several limitations:

- **Privacy regulations** such as GDPR and HIPAA prohibit the collection of sensitive data including medical records, financial transactions, and personal communications
- **Network bandwidth constraints** make large-scale data uploads expensive and time-consuming
- **Data sovereignty requirements** mandate that certain data remain within specific geographic regions
- **Single point of failure and security risks** also pose a significant concern, as centralizing millions of users' raw data creates an extremely lucrative target for cyberattacks; if the central server is compromised, the entire raw dataset is exposed, directly violating the core cybersecurity principle of data minimization.

Federated Learning (FL) addresses these constraints by inverting the traditional machine learning paradigm: instead of centralizing data, the model is distributed to the data sources. Each participating device (client) trains a local copy of the model using its private dataset, and then transmits only the computed model updates (e.g., weights or gradients) to a central server. The server aggregates these ephemeral updates to produce an improved global model, which is subsequently redistributed to the clients for the next training round.

This decentralized approach offers several fundamental advantages:

- **Enhanced Privacy through Data Minimization:** By ensuring that raw data never leaves the local device, FL fundamentally adheres to the principle of data minimization, drastically reducing privacy risks and the attack surface.
- **Bandwidth Efficiency:** Network bandwidth requirements are significantly reduced, as communicating focused model updates requires far less data transfer than uploading massive raw datasets to a cloud server.

- **Massive Scalability:** The system architecture is inherently designed for massive distribution, capable of scaling to accommodate millions of edge devices (cross-device FL) or collaborating across multiple institutions (cross-silo FL).

However, despite its benefits, deploying Federated Learning in real-world environments introduces unique technical challenges:

- **Statistical Heterogeneity (Non-IID Data):** Data generated across edge devices is highly personalized, unbalanced, and non-independent and identically distributed (Non-IID). This skewness can cause local models to overfit to their specific data, leading to *weight divergence*, which severely complicates global model convergence and degrades overall accuracy.

- **Communication Bottlenecks:** Although raw data is not transferred, modern deep neural networks contain millions of parameters. Uploading these dense parameter matrices over slow and asymmetric mobile networks (where uplink speeds are typically much slower than downlink) remains a critical bottleneck.

- **Systems Heterogeneity and Stragglers:** Participating devices possess vastly different hardware capabilities (CPU, memory, battery life) and operate under unstable network conditions. FL algorithms must be exceptionally resilient to tolerate "stragglers"—devices that compute slowly or drop out abruptly mid-training—without stalling the entire aggregation process.

2 Background

2.1 From Centralized Data Centers to Edge Networks

Prior to Federated Learning (FL), distributed machine learning primarily relied on *Data Parallelism* (partitioning data across multiple data-center workers) and *Model Parallelism* (partitioning the network architecture across devices). Both paradigms assume a highly controlled environment with high-bandwidth, low-latency network connections and independent and identically distributed (IID) data.

FL departs fundamentally from these centralized paradigms. It is engineered specifically for decentralized edge devices and central coordinators operating over

unpredictable wide-area networks (WANs). In terms of the OSI model, production-scale FL systems operate primarily at the Application Layer , utilizing protocols like HTTP/2. However, they heavily rely on the Transport Layer, specifically TCP, to ensure the reliable transmission of gigabytes of model weight updates over lossy networks [5]. This architecture replaces the high-speed data center interconnects of traditional distributed training with the variable nature of the public internet.

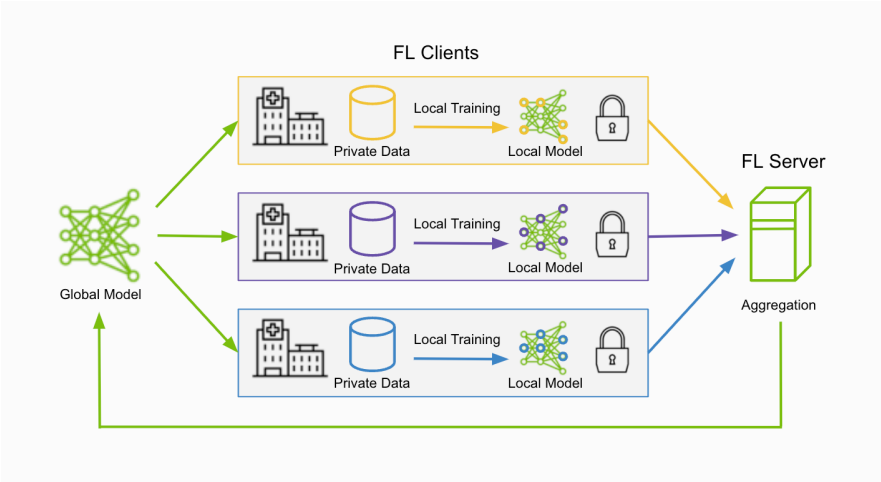

Figure 1: Overview of the Federated Learning architecture. Each client trains a local model on its private data and transmits only the updated weights to the central server, which aggregates them via FedAvg to produce an improved global model. [7]

2.2 The Federated Averaging (FedAvg) Protocol

The foundational mechanism for model aggregation over networks in FL is the Federated Averaging (FedAvg) algorithm, introduced by McMahan et al. [1]. As illustrated in Figure 1, the training process operates in iterative communication rounds over the network:

1. **Broadcast (Downlink):** The server transmits the current global model parameters over the network to a subset of available clients.

2. **Local Training:** Each client trains the received model on its private dataset for a specified number of local epochs (E) using stochastic gradient descent (SGD). This step requires no network communication.

3. **Transmission (Uplink):** Clients send only their updated weights back to the server. From a networking perspective, this is often the most significant bottleneck due to asymmetrical residential network speeds.

4. **Aggregation:** The server computes the new global model by taking a weighted average of the received updates:

$$w_{global} = \sum_{k=1}^{K} \frac{n_k}{n_{total}} w_k \tag{1}$$

where w_k represents the updated parameters from client k, and n_k denotes the number of local training samples.

3 Proposed Approach and Technical Analysis

3.1 Network Architecture and Protocol Stack

Unlike traditional web requests where payloads are relatively small (e.g., HTML files or JSON responses), Federated Learning (FL) involves the transmission of dense mathematical structures—specifically, deep neural network weights or gradients. A modern Large Language Model (LLM) or even a standard ResNet computer vision model can range from tens of megabytes to several gigabytes in size.

To handle this, production FL frameworks typically employ a robust protocol stack. At the Application Layer, Remote Procedure Calls over HTTP/2 are utilized to manage the complex, bidirectional streaming of parameters. This sits on top of the Transport Layer, where the Transmission Control Protocol (TCP) is mandatory. The reliable, ordered, and error-checked delivery provided by TCP is critical; a single dropped UDP datagram would corrupt the entire model update, rendering the client's computational effort useless. Furthermore, Transport Layer Security (TLS) is encapsulated within this flow to prevent man-in-the-middle (MitM) attacks from intercepting or poisoning the model weights.

3.2 Control Logic and Packet Flow in a Training Round

The network lifecycle of a single FL communication round involves several distinct phases, each with specific network characteristics:

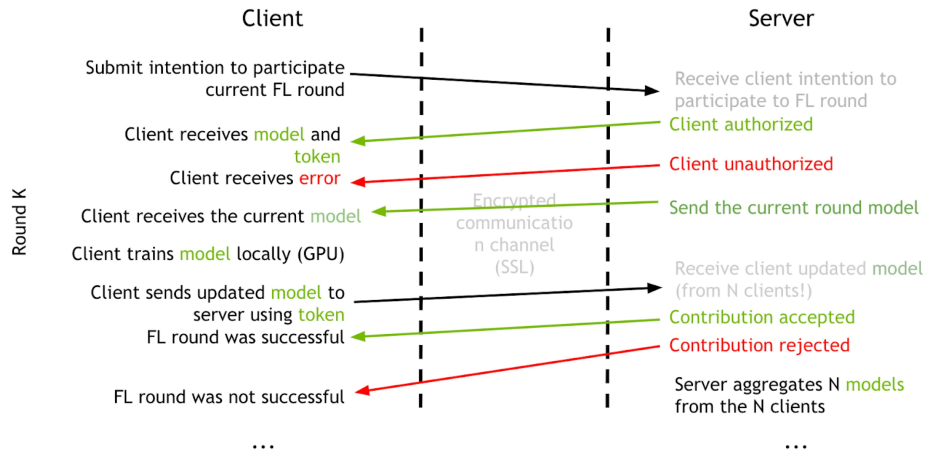

Figure 2: Communication sequence between client and server during a single FL round (Round K). The encrypted channel (SSL/TLS) handles model broadcast and update transmission [8].

1. **Connection Establishment:** The client initiates a standard 3-way TCP handshake (*SYN, SYN-ACK, ACK*) with the central server, followed by a TLS handshake. Production FL frameworks additionally implement a token-based authorization mechanism, whereby the server issues a session token alongside the global model; clients must present this token when submitting updates, preventing unauthorized contributions to the aggregation. [8]

2. **Client Selection & Broadcast (Downlink):** The server selects a subset of available clients. It then streams the global model weights. From a network traffic perspective, this phase saturates the server's upload bandwidth and the clients' download bandwidth.

3. **Silent Computation Phase:** The network connection remains idle (or relies on periodic TCP keep-alive packets) while the client performs Local Training (SGD) using its local GPU/CPU. This period can last from minutes to hours depending on the device's compute capability.

4. **Update Transmission (Uplink):** The client transmits the calculated delta (weight updates) back to the server. This is typically the most vulnerable phase of the packet flow due to network instability.

5. **Connection Termination:** Once the server acknowledges (*ACK*) the receipt of the full payload, the connection may be gracefully terminated (*FIN, FIN-ACK*) to free up server sockets.

Figure 2 provides a complete sequence diagram of a single FL round (Round K), illustrating the authorization flow, encrypted model broadcast, local training phase, and the contribution acceptance/rejection outcomes that map directly to the packet flow phases described above.

3.3 Inherent Weaknesses and Scalability Limits

While FL solves data privacy issues, it shifts the burden directly onto the network infrastructure, introducing significant scalability limits.

- **Asymmetric Bandwidth Bottlenecks:** Consumer ISPs provision networks asymmetrically: download speeds are abundant, but uplink capacity is severely constrained. While a client can receive a global model with ease, transmitting the same volume of updated weights back to the server causes edge congestion that compounds across thousands of simultaneous participants. Konečný et al. [3] address this directly by proposing *structured updates*, where clients transmit low-rank or randomly masked approximations of the full gradient rather than the complete parameter matrix, and *sketched updates*, which apply quantization and subsampling after computing the full local update. These strategies reduce uplink communication costs by up to two orders of magnitude, making the asymmetric bottleneck tractable without sacrificing model quality.

- **Concurrent Connection Limits (The C10k Problem):** In a global FL deployment, a server must coordinate millions of devices simultaneously. Maintaining long-lived TCP connections during the silent computation phase—where sockets are held open while clients train locally—places enormous memory and scheduling pressure on the server. Kairouz et al. [2] identify this as a fundamental open problem, drawing a sharp distinction between *cross-device* FL, where millions of mobile endpoints participate under volatile network conditions, and *cross-silo* FL, where a smaller number of institutional nodes operate over stable, high-bandwidth links. The

two settings demand entirely different infrastructure designs, yet both ultimately confront the same underlying tension between connection scalability and aggregation correctness.

- **The Straggler Problem:** FedAvg is synchronous by design: the server waits to collect updates from all selected clients before aggregating. A single client on a congested or low-bandwidth connection can therefore stall an entire round. Servers mitigate this by broadcasting to a larger cohort and proceeding once a sufficient fraction responds— discarding the remainder—but this introduces selection bias into the aggregation. Li et al. [4] formalize this as *systems heterogeneity* and propose FedProx, which adds a proximal regularization term to the local objective. This allows each device to perform a variable amount of local work proportional to its available compute and network resources, rather than enforcing uniform epochs across the cohort, demonstrating how tightly network conditions and optimization outcomes are coupled.

- **Statistical Heterogeneity and Its Network Implications:** Non-IID data distribution across clients causes local models to drift away from the global optimum during training—a phenomenon known as *weight divergence*. Zhao et al. [6] show that FedAvg accuracy degrades sharply as data heterogeneity increases, and that the most effective mitigation is sharing a small globally representative data subset across clients. From a network perspective, this remedy is self-defeating: it reintroduces raw data transfer over the uplink, directly undermining the bandwidth efficiency that motivates FL. This reveals a deeper tension—statistical and systems heterogeneity are not independent problems but interact through the network layer in ways that resist simple solutions.

- **Gradient Compression Trade-offs:** A natural response to uplink bottlenecks is to compress model updates before transmission. Konečný et al. [3] propose two families of compression: *structured updates*, which constrain the update to a low-rank or randomly masked subspace, and *sketched updates*, which apply quantization and subsampling to a full gradient before sending. Both approaches reduce uplink costs by up to two orders of magnitude; however, compression introduces a fundamental trade-off—aggressive quantization degrades convergence speed, meaning fewer bits per round translates directly into more rounds required, shifting the bottleneck from bandwidth to latency.

- **Communication Round Complexity:** Unlike centralized training where a single forward-backward pass updates the model, FL requires multiple full communication rounds—each involving a complete broadcast and uplink cycle—to reach comparable accuracy. McMahan et al. [1] show that increasing local computation per round (larger E) reduces the total number of rounds needed, but Kairouz et al. [2] note that this gain saturates quickly under non-IID data, since excessive local training causes client models to diverge from the global objective. The number of communication rounds therefore represents a hard network cost that cannot be arbitrarily reduced without degrading model quality.

- **Cross-Device vs. Cross-Silo Scalability Gap:** Kairouz et al. [2] draw a sharp architectural distinction between two FL deployment regimes. In *cross-silo* FL, a small number of institutional nodes—hospitals, banks, data centers—communicate over stable, high-bandwidth links with predictable availability. In *cross-device* FL, millions of mobile endpoints participate under volatile network conditions, with each device available only intermittently. These two regimes demand fundamentally different protocol designs: cross-silo systems can afford synchronous aggregation and persistent connections, while cross-device systems require asynchronous protocols, aggressive compression, and tolerance for high dropout rates. Conflating the two leads to systems that are over-engineered for one setting and entirely unsuitable for the other.

3.4 Failure Scenarios and Fault Tolerance Mechanisms

Edge networks—Wi-Fi, 4G/5G—are inherently unreliable. FL systems must tolerate failure at every phase of the communication round. Figure 3 depicts the server-side state machine governing a complete training round. Timeout transitions from both the 'Wait for clients' and 'Wait for updates' states trigger graceful shutdown or administrator notification, directly implementing the straggler and dropout mitigation mechanisms described in this section.

- **Drop-outs and Network Partitions:** When a client loses connectivity mid-uplink, its in-flight weight update is lost and the TCP connection drops. The server must recover gracefully, discarding the partial payload and relying on the remaining cohort. Kairouz et al. [2] frame client availability as a first-class systems challenge: in cross-device FL, participant sets shift continuously across rounds due to battery constraints, mobility, and network

interruptions. Aggregation algorithms must therefore remain unbiased under arbitrary dropout patterns, a requirement that standard FedAvg does not formally satisfy.

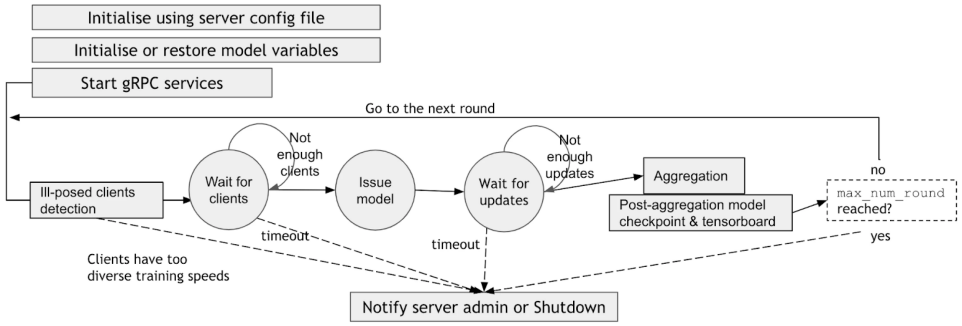

Figure 3: Server-side state machine governing a federated training round. Timeout transitions from the *Wait for clients* and *Wait for updates* states trigger graceful shutdown or administrator notification, implementing straggler and dropout fault tolerance [8].

- **Partial Updates and Convergence Under Failure:** Li et al. [4] show that the proximal term introduced by FedProx provides convergence guarantees even when clients submit incomplete local updates due to timeout or disconnection. By bounding the divergence between local and global objectives, FedProx ensures that truncated contributions from straggling clients do not destabilize the global model—a critical property for any production deployment where network failure is not the exception but the norm.

- **The "Thundering Herd" Problem:** When thousands of clients complete local training simultaneously and attempt to upload at the same instant, the resulting burst of concurrent TCP connections and large payloads can overwhelm the server's network interface—a failure mode behaviorally identical to a volumetric DDoS attack. Bonawitz et al. [5] document this in production and propose *pace steering*: clients are assigned randomized check-in windows that distribute upload traffic over time. Their analysis emphasizes that client scheduling, server socket management, and aggregation logic must be co-designed as a single system, since optimizing any one component in isolation is insufficient to prevent throughput collapse under load.

- **Security Threats at the Network Layer:** FL's architecture also introduces

deliberate attack surfaces. Model updates traversing the network can be intercepted by a man-in-the-middle adversary and manipulated—a class of attack known as *model poisoning*. Kairouz et al. [2] distinguish between *Byzantine clients* that inject malicious updates and passive adversaries that attempt to reconstruct private training data from intercepted gradients. The former requires robust aggregation to detect and down-weight anomalous submissions; the latter necessitates TLS encryption of all parameter transmissions. These are not optional additions to the protocol stack but baseline requirements for any deployment over a public network.

- **Byzantine Fault Tolerance and Model Poisoning:** FL's distributed update mechanism introduces a deliberate attack surface: a malicious client can submit a carefully crafted update designed to corrupt the global model—a *Byzantine* failure. Unlike a network dropout, which is detectable from a missing TCP payload, a poisoned update is structurally valid and indistinguishable from a legitimate gradient without additional verification. Kairouz et al. [2] identify Byzantine robustness as one of the most critical open problems in FL, noting that standard FedAvg weighted averaging provides no protection, since a single malicious update is incorporated into the global model in direct proportion to the attacker's reported dataset size.

- **Secure Aggregation Protocol:** Even without active adversaries, model updates transmitted in plaintext over a public network expose gradient information that can be used to reconstruct private training data. Bonawitz et al. [5] address this at the protocol level through *secure aggregation*: a cryptographic protocol in which the server learns only the *sum* of client updates, never any individual contribution. Clients exchange pairwise secret masks that cancel in the aggregate, ensuring that gradient privacy is preserved even if the server is honest-but-curious. From a network perspective, secure aggregation adds a coordination round prior to the uplink phase, introducing additional latency but providing a formal privacy guarantee that TLS encryption alone cannot offer.

4 Conclusion

Federated Learning reframes network communication as the primary engineering constraint in distributed machine learning. Rather than optimizing for compute efficiency, FL systems must co-design their optimization algorithms, protocol stack,

and fault tolerance mechanisms around asymmetric bandwidth, volatile connectivity, and heterogeneous client hardware. The challenges analyzed here—the straggler problem, weight divergence under non-IID data, and Byzantine vulnerability—are not implementation deficiencies but structural consequences of decentralized training over wide-area networks.

FL is the superior architectural choice under three specific conditions: when raw data cannot leave the device due to regulatory constraints such as GDPR or HIPAA; when uplink bandwidth is scarce relative to dataset size, making centralized collection impractical; and when the participant population is large enough that FL's communication overhead is outweighed by the cost of centralizing data at scale. Outside these conditions, centralized training remains simpler and more efficient.

5 Python-Based FL Implementation

To validate the theoretical analysis presented in this report, a Python-based Federated Learning system was implemented using raw TCP sockets and PyTorch. The implementation consists of three components: a central server (`server.py`), a client (`client.py`), and shared utilities (`utils.py`).

Model and Data Distribution

The model used across all experiments is a simple three-layer fully connected network (SimpleNN: 784→128→64 →10), trained on the MNIST dataset. To simulate realistic FL conditions, data was distributed across clients in a non-IID fashion by assigning each client a skewed subset of digit classes, where some classes are over-represented relative to others, partially replicating the statistical heterogeneity discussed in Section 3.3. Each client performs two local epochs per round before serializing and transmitting its updated weights to the server via TCP. Aggregation follows the standard FedAvg weighted average, with each client's contribution weighted by its local dataset size.

Experimental Setup

Two configurations were evaluated over 10 communication rounds: 3 and 10 clients. The server broadcasts the global model at the start of each round, collects all client updates, aggregates via FedAvg, and evaluates the resulting global

model on a held-out MNIST test set. Network traffic was captured using Wireshark throughout both experiments to observe real TCP-level communication behavior.

Results and Discussion

The results are summarized in Table 1. The 3-client configuration achieves the highest final accuracy (93.65%) and converges fastest, reaching above 90% by round 5. This is consistent with the theoretical expectation: fewer clients with larger per-client data shares produce more representative local updates, reducing weight divergence during aggregation. The 10-client configuration converges more slowly and plateaus at a lower accuracy (89.08%), reflecting the increased statistical heterogeneity as data is partitioned across more clients with narrower digit class coverage. Notably, both configurations exhibit a small accuracy fluctuation around rounds 6–8 before stabilizing, a characteristic oscillation caused by the tension between local overfitting and global averaging under non-IID conditions.

Table 1: Global model accuracy (%) over 10 FL rounds across two client configurations.

Round	**3 Clients**	**10 Clients**
1	22.77	50.96
2	71.69	71.68
3	78.58	78.02
4	86.65	79.70
5	91.38	83.88
6	91.12	84.84
7	91.63	86.43
8	91.77	87.35
9	92.99	88.80
10	93.65	89.08

As shown in Figures 4 and 5, the 3-client run generated approximately 26 MB of total TCP traffic over 12 minutes and 29 seconds at an average throughput of 281 kbps. The 10-client run produced 88 MB over 9 minutes and 34 seconds at 1,227 kbps, reflecting the linear scaling of model broadcast and update collection with client count. In both cases, each TCP conversation transferred approximately 4 MB in each direction per round, corresponding to the serialized

SimpleNN parameter payload. The symmetric bidirectional transfer (Bytes A→B ≈ Bytes B→A) confirms that the implementation operates on loopback without the asymmetric uplink penalty that characterizes real-world deployments, as discussed in Section 3.3.

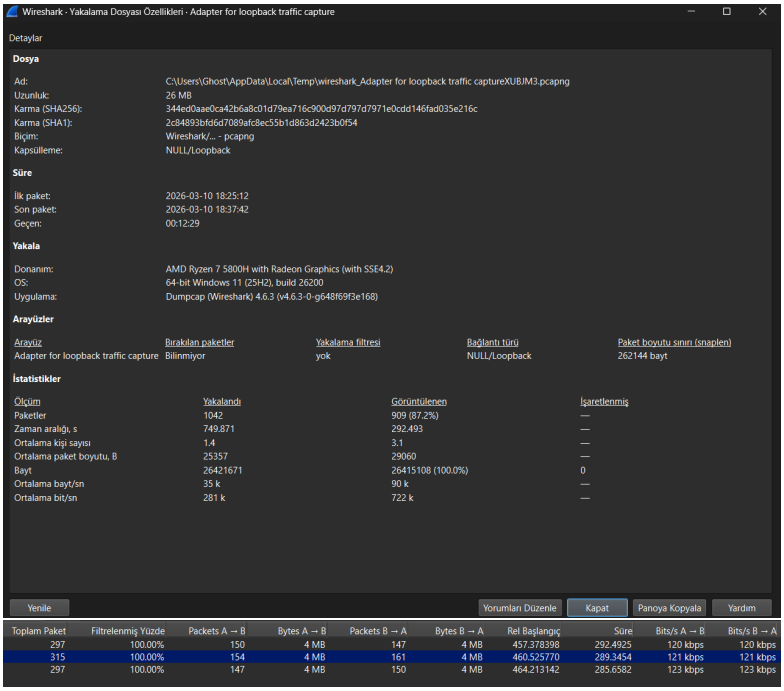

Figure 4: Wireshark capture for the 3-client experiment: capture file properties (top) and TCP conversation statistics (bottom). Total traffic: 26 MB over 12m 29s at 281 kbps average.

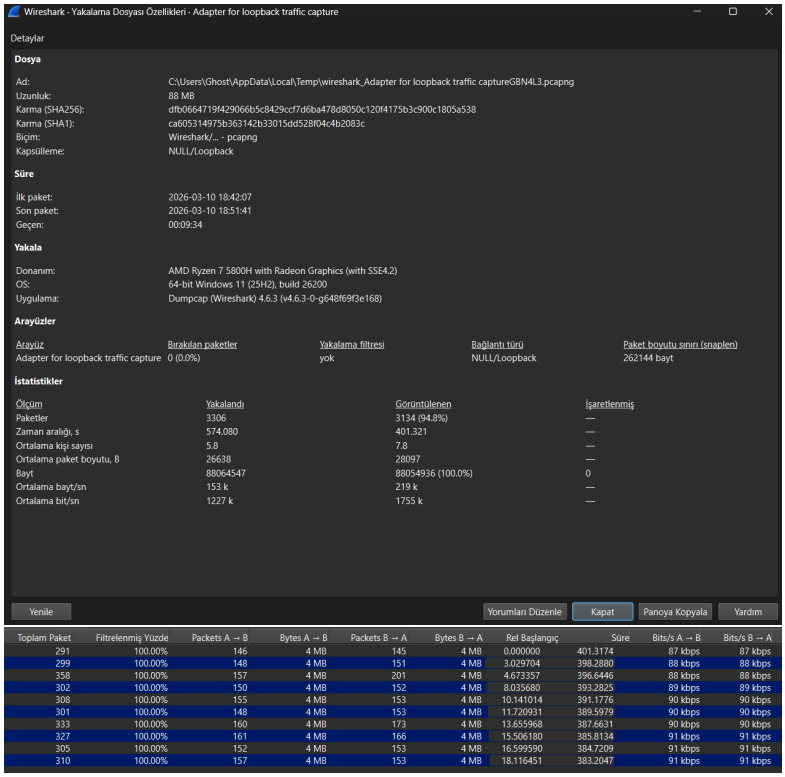

Figure 5: Wireshark capture for the 10-client experiment: capture file properties (top) and TCP conversation statistics (bottom). Total traffic: 88 MB over 9m 34s at 1,227 kbps average.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," 2023, arXiv:1602.05629.

https://arxiv.org/abs/1602.05629

[2] P. Kairouz et al., "Advances and Open Problems in Federated Learning," 2021, arXiv:1912.04977.

https://arxiv.org/abs/1912.04977

[3] J. Konečný et al., "Federated Learning: Strategies for Improving Communication Efficiency," 2017, arXiv:1610.05492.

https://arxiv.org/abs/1610.05492

[4] T. Li et al., "Federated Optimization in Heterogeneous Networks," 2020, arXiv:1812.06127.

https://arxiv.org/abs/1812.06127

[5] K. Bonawitz et al., "Towards Federated Learning at Scale: System Design," 2019, arXiv:1902.01046.

https://arxiv.org/abs/1902.01046

[6] Y. Zhao et al., "Federated Learning with Non-IID Data," 2018, arXiv:1806.00582.

https://doi.org/10.48550/arxiv.1806.00582

[7] https://nvflare.readthedocs.io/en/2.4/fl_introduction.html

[8] https://docs.nvidia.com/clara/clara-train-archive/3.1/federated-learning/fl_background_and_arch.html